T15 Gestion des Secrets MCP

Technique Advanced

Gérer les credentials des MCP servers sans les exposer

Le problème central

Les MCP servers ont besoin de clés API et de credentials pour fonctionner. Les stocker en clair dans `settings.json` ou `.mcp.json` crée un risque réel : commit accidentel sur Git, exposition dans les logs, ou mouvement latéral après une compromission. La règle absolue est que les secrets ne doivent jamais apparaître dans les fichiers versionnés.

Trois approches selon le contexte

Approche	Sécurité	Complexité	Idéal pour
OS Keychain	Élevée	Moyenne	Devs solo, macOS/Linux
`.env` + `.gitignore`	Moyenne	Faible	Petites équipes, prototy-page
Secret Vaults	Très élevée	Haute	Enterprise, SOC 2, HI-PAA

Approche 1 : OS Keychain (recommandée)

Le secret est chiffré au repos par l'OS et accessible uniquement aux processus autorisés. La configuration MCP n'expose qu'une commande de récupération, jamais la valeur.

```
# Exemple macOS : les vraies valeurs saisies ici peuvent apparaître dans l'historique shell
# Préférer l'interface interactive Trousseaux d'accès pour les secrets réels
security add-generic-password \
  -a "claude-mcp" -s "github-token" \
  -w "ghp_votre_token_ici"
```

```
{
  "mcpServers": {
    "github": {
      "command": "bash",
      "args": ["-c", "GITHUB_TOKEN=$(security find-generic-password -s 'github-token' -w) /path/to/approved-mcp-server"]
    }
  }
}
```

Linux : utiliser `secret-tool` (libsecret, GNOME Keyring ou KWallet) avec la même logique de wrapper script.

Approche 2 : .env + .gitignore

Simple et suffisante pour un usage solo ou une petite équipe avec bonne discipline.

```
# Créer le fichier de secrets
cat > ~/.claude/.env << EOF
GITHUB_TOKEN=ghp_votre_token
OPENAI_KEY=sk-votre_cle
EOF
chmod 600 ~/.claude/.env   # Permissions restrictives
echo ".env" >> ~/.claude/.gitignore
```

Claude Code ne charge pas automatiquement ce fichier `.env`. Exporter les variables requises via un lanceur de confiance ou un gestionnaire de secrets avant le démarrage, puis référencer `"${GITHUB_TOKEN}"` dans le champ MCP `env`. Relire un fichier shell avant de le sourcer.

Template pour les équipes : commiter `mcp-config.template.json` avec des placeholders, générer le fichier réel via `envsubst`. Ne jamais commiter le fichier généré.

Approche 3 : Secret Vaults (enterprise)

HashiCorp Vault, AWS Secrets Manager, ou 1Password CLI permettent une rotation automatisée, un audit centralisé et un contrôle d'accès granulaire. Le principe reste identique : un wrapper script récupère le secret au runtime et l'exporte comme variable d'environnement avant de lancer le server MCP.

```
# HashiCorp Vault
export GITHUB_TOKEN=$(vault kv get -field=token secret/claude/github)
/path/to/approved-mcp-server
```

Ce qu'il ne faut jamais faire

Hardcoder un token directement dans la valeur `command` ou `args` d'un MCP server versionné. Même dans un repo privé, les secrets en clair dans Git restent dans l'historique même après suppression et peuvent être exposés par un `git log` ou un accès non autorisé.

Un hook de pre-commit qui scanne les patterns de tokens (`ghp_`, `sk-`, `Bearer`) avant chaque commit est une protection supplémentaire simple à mettre en place.

`CLAUDE_CODE_SUBPROCESS_ENV_SCRUB` **(v2.1.83+)** : retire les variables sensibles reconnues par Claude Code, dont les identifiants Anthropic/cloud et ceux présents dans les URL de registres de paquets, avant transmission aux sous-processus et serveurs MCP. Ce n'est pas un détecteur universel de noms de secrets ; il peut casser les sous-processus qui en ont besoin. Vérifier l'environnement effectif.